

“Año de la Esperanza y el Fortalecimiento de la Democracia”



CURSO: Arquitectura Orientada al Servicio

PROYECTO: Implementación de un Sistema de Tickets Automatizado

PROFESOR: Ing. Ronald Angel Ayra Pinto

Estudiantes:

Flores Mamani Jorge U22212995

Villar Cruz Juan U19207007

Rivera Galdos Luis Angel U23267620

Hurtado Pasache Josep U18309515

Flores Ramirez Anthony Guiliano U23100394

Ramos Rosales Jared Joel U20213721

Hernández Villafuerte Nicole U22233439

Perú, 2026

Contents

PROYECTO: ARQUITECTURA ORIENTADA AL SERVICIO	2
AVANCE N.º 2 – DISEÑO INICIAL DEL SERVICIO WEB	2
1. INTRODUCCIÓN	2
2. DEFINICIÓN DEL SISTEMA DE SERVICIO WEB	2
2.1 Sistema de servicio web a desarrollar	3
2.2 Problemática	3
2.3 Solución propuesta	3
2.4 Lenguajes y tecnologías a utilizar	4
3. PARTES DEL SERVICIO WEB.....	5
3.1 Contrato.....	5
3.2 Interfaz.....	6
3.3 Implementación	8
3.4 Lógica de negocio	8
3.5 Datos	9
4. TIPOS DE SERVICIOS	9
4.1 Servicio de utilidad.....	10
4.2 Servicio de entidad.....	10
4.3 Servicio de tarea	11
5. SEGURIDAD DEL SERVICIO WEB	11
5.1 Autenticación	11
5.2 Autorización	12
5.3 Confidencialidad	12
5.4 Disponibilidad	13
6. Arquitectura de Software en Tiempo de Ejecución	13
6.1 Arquitectura Rígida	13
Configuración del backend y servicios REST	14
Estructura de la base de datos	14
Configuración de seguridad	14
Infraestructura del servidor	14
6.2 Arquitectura Flexible.....	15
Gestión dinámica de tickets	15

Administración de usuarios y roles	15
Monitoreo y dashboards en tiempo real	15
Notificaciones automáticas	16
Escalabilidad operativa	16
7. Modelos de Distribución y Licenciamiento	16
8. Diagramas Técnicos	17
Fase 1: Captura e Inicio de la Petición (Mensajes 1 y 2).....	18
Fase 2: Filtro de Seguridad y Autenticación (Mensajes 3 y Envoltorio critical)	18
Fase 3: Procesamiento y Lógica de Negocio (Mensajes 4, 5, 6 y Envoltorio rect) ..	18
Fase 4: Persistencia y Respuesta Final (Mensajes 7 al 10).....	19
1. Petición y Control de Acceso (Pasos 1 al 3)	19
2. Procesamiento de Lógica de Negocio (Paso 4 y Auto-llamada)	20
3. Persistencia en la Base de Datos (Pasos 5 y 6)	20
4. Procesos Internos Automatizados (Bloque rect - Pasos 7 y 8)	20
5. Finalización del Flujo.....	20
1. Capa de Presentación (Frontend React)	24
2. Capa de Contratos (Controllers REST - Spring Boot)	24
3. Capa de Lógica de Negocio (Services)	25
4. Capa de Persistencia y Datos	25
9. Estado de Avance del Servicio Web (50%)	26
10. CONCLUSIONES.....	30
11. BIBLIOGRAFÍA.....	30

1. INTRODUCCIÓN

En el ámbito del desarrollo de software moderno, la Arquitectura Orientada al Servicio (SOA) se ha vuelto esencial para crear sistemas que operen bien a gran escala. En vez de considerar las apps como elementos aislados, SOA facilita comprender el software como un grupo de servicios autónomos que se conectan entre ellos. Este modo de crear sistemas es crucial hoy, pues ayuda a que las empresas no se atasquen en tecnologías antiguas, dándoles la opción de renovar, juntar y expandir sus herramientas digitales de forma rápida y sin problemas técnicos graves. Bajo este enfoque de modernización, el presente avance documenta el inicio del proyecto de implementación de un Sistema de Tickets Automatizado para la Mesa de Ayuda de SERPOST.

El objetivo principal es transformar el modelo actual, donde el registro manual de incidencias por parte de los técnicos genera cuellos de botella y errores operativos. En estas páginas, detallamos desde el análisis del contexto empresarial y el diagnóstico del problema, hasta la planificación estratégica bajo el framework Scrum y el diseño de los requerimientos de software. Con esto, buscamos establecer una base técnica sólida, utilizando herramientas como Java y MySQL, para mejorar la trazabilidad de las solicitudes y, sobre todo, elevar la satisfacción del usuario final.

2. DEFINICIÓN DEL SISTEMA DE SERVICIO WEB

2.1 Sistema de servicio web a desarrollar

- Nombre del sistema: Sistema de gestión de tickets basado en servicios web para la optimización de la mesa de ayuda en SERPOST

Este sistema está diseñado para modernizar la atención técnica en organizaciones con infraestructuras logísticas o administrativas complejas, como SERPOST, permitiendo que tanto el personal administrativo como los técnicos de soporte gestionen incidencias de manera eficiente. La plataforma centraliza el flujo de trabajo mediante un modelo de autogestión donde se procesa información crítica sobre el ciclo de vida de los tickets, niveles de servicio (SLA), autenticación de usuarios y métricas de desempeño. Al implementarse como un servicio web, facilita la trazabilidad y la resolución oportuna de requerimientos tecnológicos, eliminando la dependencia de registros manuales y mejorando la operatividad global del departamento de TI.

2.2 Problemática

Actualmente, el área de mesa de ayuda del departamento de tecnología de SERPOST enfrenta una gestión ineficiente de sus servicios debido a un flujo operativo basado en el registro manual por parte del técnico. En este modelo, el usuario final no tiene una vía directa para reportar sus problemas, obligando al personal de soporte a actuar como intermediario para ingresar cada incidencia al sistema. Esta forma de trabajo es incorrecta para los estándares modernos de TI, ya que consume tiempo valioso del personal especializado en tareas administrativas básicas, en lugar de enfocarse en la resolución técnica de los problemas. Esta situación genera dificultades críticas que justifican la necesidad de un cambio:

- **Lentitud y retrasos operativos:** Existe un desfase temporal considerable entre el momento en que ocurre la falla y el momento en que el técnico logra registrarla, lo que retrasa el inicio de la atención y afecta el cumplimiento de los tiempos de respuesta.
- **Inexactitud y pérdida de información:** Al depender de la transcripción del técnico, se producen omisiones frecuentes o registros incompletos de las solicitudes, lo que dificulta el diagnóstico preciso de las fallas.
- **Falta de trazabilidad y transparencia:** El usuario final no tiene visibilidad sobre el avance de su requerimiento ni una participación activa en el proceso, lo que genera incertidumbre y una baja percepción de la calidad del servicio.

- **Sobrecarga y errores humanos:** La centralización del registro en una sola figura aumenta el riesgo de errores por fatiga y genera cuellos de botella que impiden una gestión escalable de las incidencias tecnológicas de la empresa.

2.3 Solución propuesta

La solución planteada consiste en la implementación de una plataforma web de gestión de tickets que permita a los usuarios finales registrar directamente sus propias incidencias y solicitudes mediante un acceso seguro con usuario y contraseña. Este sistema transformará el modelo de atención actual, eliminando la intermediación del técnico para el ingreso de datos y asegurando que cada requerimiento sea capturado de forma íntegra y oportuna desde su origen.

Procesos que se automatizarán:

- **Registro de incidencias y solicitudes:** Los usuarios podrán ingresar sus tickets de manera autónoma a través de la plataforma web, eliminando el registro manual por parte del técnico.
- **Notificaciones y alertas:** El sistema enviará correos electrónicos automáticos para informar sobre eventos del ticket y generar alertas cuando un requerimiento esté próximo a vencer según su nivel de prioridad.
- **Control de Niveles de Servicio (SLA):** Se integrará un motor de reglas que asigne tiempos de atención y resolución automáticos basados en la criticidad del incidente (crítico, medio y bajo o solicitud).
- **Monitoreo de rendimiento:** Generación de dashboards automáticos que permitan visualizar en tiempo real el estado general de los tickets y el cumplimiento de los indicadores de gestión.
-

Beneficios que se obtendrán:

- **Rapidez y eficiencia:** Se optimizan los tiempos de respuesta al eliminar la espera para el registro manual y se reduce drásticamente la carga operativa del personal de soporte.
- **Transparencia y trazabilidad:** El usuario obtiene visibilidad total sobre el ciclo de vida de su requerimiento, pudiendo consultar el estado de sus tickets en cualquier momento.
- **Seguridad y control:** La implementación de módulos de autenticación y gestión de cuentas por área garantiza que solo el personal autorizado acceda a la información y realice gestiones en el sistema.

- **Integración y monitoreo:** La centralización de la información en una base de datos MySQL permite medir la tasa de resolución y el cumplimiento de SLA, facilitando la toma de decisiones basada en datos reales.

2.4 Lenguajes y tecnologías a utilizar:

Lenguaje de programación

- Java
- JavaScript

Tipo de servicio

- REST

Base de datos

- MYSQL

Frameworks

- Spring Boot
- React

Formato de intercambio de datos

- JSON

Para el desarrollo del sistema propuesto, se ha seleccionado Java como lenguaje de programación para el backend, debido a su solidez, seguridad y amplia adopción en entornos empresariales, lo que lo convierte en una opción adecuada para sistemas de gestión. Asimismo, se utilizará el framework Spring Boot para la construcción del servicio web, ya que permite estructurar aplicaciones bajo una arquitectura REST de manera eficiente, facilitando la creación de endpoints, la gestión de dependencias y la integración con bases de datos. En el frontend, se empleará React, lo cual permitirá desarrollar interfaces de usuario dinámicas, reutilizables y altamente interactivas, mejorando significativamente la experiencia del usuario en el registro y seguimiento de tickets. El servicio se implementará bajo el estilo REST debido a su simplicidad, escalabilidad y compatibilidad con aplicaciones web modernas. Por otro

lado, se utilizará MySQL como sistema de gestión de base de datos, gracias a su capacidad para manejar información estructurada de forma segura y eficiente, garantizando la integridad de los datos. Finalmente, se adoptará JSON como formato de intercambio de datos entre el cliente y el servidor, por ser ligero, fácil de interpretar y ampliamente utilizado en servicios web.

3. PARTES DEL SERVICIO WEB

3.1 Contrato

El contrato del servicio web define los mecanismos mediante los cuales los clientes interactúan con el sistema, especificando los métodos HTTP y los endpoints disponibles.

Métodos HTTP

Se utilizarán los siguientes métodos:

- GET: Para la consulta de información, como la visualización de tickets.
- POST: Para el registro de nuevos datos, como la creación de tickets o usuarios.
- PUT: Para la actualización de información existente, como el estado o asignación de un ticket.
- DELETE: Para la eliminación de registros dentro del sistema, según los permisos establecidos.

Endpoints principales:

Los endpoints estarán organizados en función de las funcionalidades del sistema:

- /api/tickets → Permite registrar y consultar tickets
- /api/tickets/{id} → Permite consultar y actualizar un ticket específico
- /api/usuarios → Permite la gestión de usuarios
- /api/login → Permite la autenticación de usuarios
- /api/dashboard → Permite obtener información para la visualización del estado general del sistema

Herramienta de documentación:

Se utilizará Swagger para la documentación del servicio web, lo que permitirá describir de manera estructurada los endpoints, métodos y parámetros, además de facilitar su prueba e integración.

3.2 Interfaz

La interfaz del servicio web define las operaciones disponibles para el cliente, permitiendo la interacción con el sistema.

Operaciones principales

Registrar:

Permite a los usuarios crear nuevos tickets, ingresando información relevante como descripción, tipo de incidencia y prioridad.

Consultar:

Permite visualizar los tickets registrados, así como su estado y detalles asociados.

Actualizar:

Permite modificar la información de los tickets, como su estado, prioridad o asignación a un técnico.

Eliminar:

Permite eliminar registros del sistema, de acuerdo con los permisos definidos para cada tipo de usuario.

La interfaz permite la comunicación entre el cliente y el servicio web, mediante el envío de solicitudes HTTP y la recepción de respuestas en formato JSON. Esto asegura una arquitectura desacoplada, escalable y alineada con los principios de servicios web.

Balsamiq Wireframe

[←](#)
[→](#)

SERPOST - Mesa de Ayuda TI

Label

Usuario

Contraseña

Ingresar

Label

REPORTAR NUEVO REQUERIMIENTO

Tipo

Incidente/Solicitud

Categoría

Software/Hardware/Red

Asunto

Descripción

Attach file/screenshot

ENVIAR TICKET

Balsamiq Wireframe

[←](#)
[→](#)

Vertical Tabs

Bandeja de Entrada

Mis Tickets

SLAs

Reportes

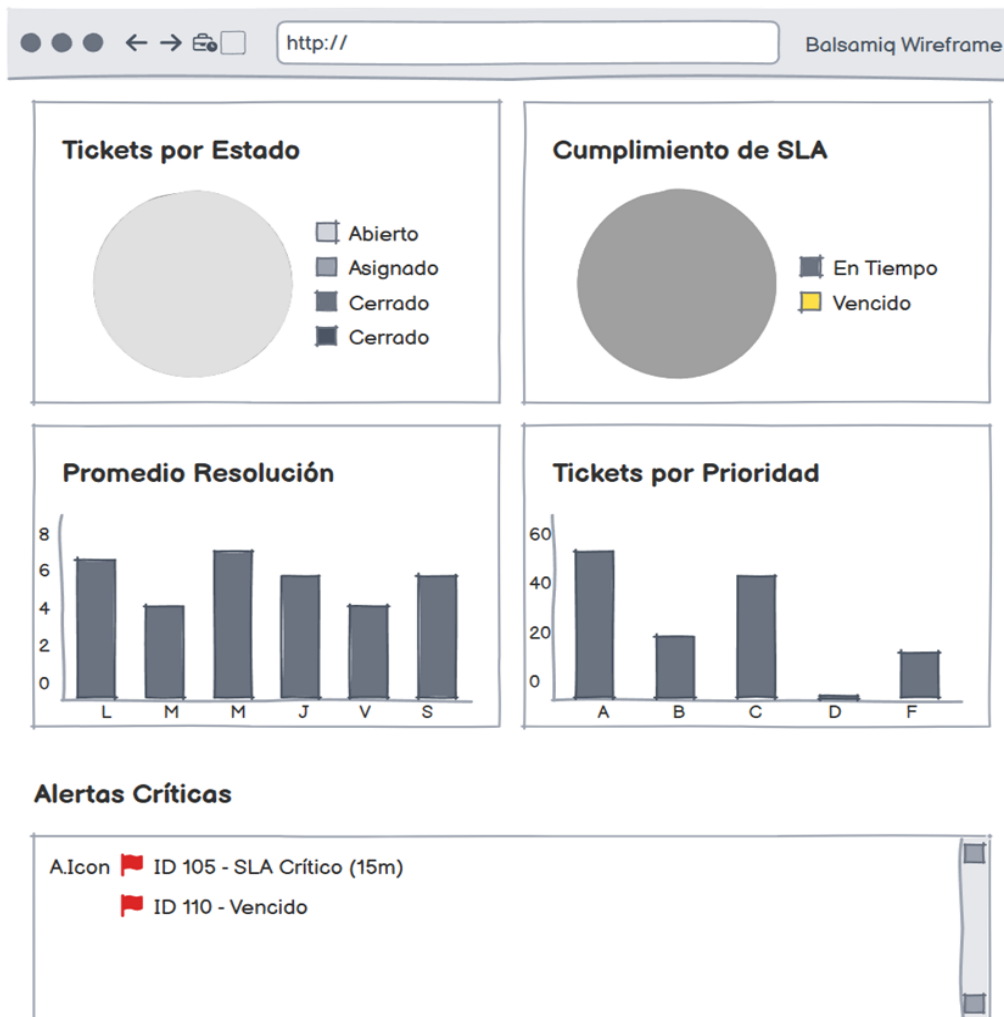
Search Bex

Filtros

Combo Box

Data Grid

ID	Usuario	Asunto	Categoría	Estado	SLA Restante	Acciones
			Cerrado	●	1h 30m	✕
			Asignado	●	4h 15m	✕
			Hardware	●	4h 15m	✕
			Hardware	●	4h 15m	✕
			Software	●	4h 15m	✕
			Cerrado	●	4h 15m	✕



3.3 Implementación

El sistema se implementará utilizando Java como lenguaje principal en el backend, ya que, al ser un lenguaje muy robusto y tipado, apoyado en el framework Spring Boot, nos permitirá construir servicios REST de manera modular y escalable. El frontend se desarrollará con JavaScript, con el framework de React, el cual facilitará la creación de interfaces dinámicas y responsivas para los usuarios finales. La base de datos será MySQL, encargada de almacenar la información de tickets y usuarios. En esta sección se documentará el código fuente en futuras etapas del proyecto, incluyendo ejemplos de controladores, servicios y componentes de interfaz.

3.4 Lógica de negocio

La capa de lógica de negocio definirá las reglas y procesos internos del sistema, sin acceder directamente a la base de datos. Entre las principales reglas se incluyen:

a) Validaciones:

- Verificar que los campos obligatorios del ticket (usuario, descripción, prioridad) estén completos, así como que la información sea coherente.
- Validar que el estado del ticket siga un flujo correcto (ejemplo: Abierto → En proceso → On Hold → Resuelto → Cerrado).

b) Condiciones:

- Solo el personal autorizado podrá cambiar el estado de un ticket.
- Los usuarios finales podrán crear y consultar sus propios tickets, pero no modificarlos una vez registrados.

c) Procesos internos:

- Asignación automática de tickets según categoría o nivel de prioridad.
- Notificación al usuario cuando su ticket cambie de estado.
- Registro de historial de acciones para cada ticket.

3.5 Datos

El sistema almacenará información estructurada en distintas entidades relacionadas:

- Clientes: datos de contacto y empresa (teléfono, dirección, razón social).
- Usuarios: información de los usuarios del sistema, incluyendo nombre, correo, credenciales y rol (administrador, técnico, cliente).
- Tickets: registros de incidencias con título, descripción, fecha de creación, estado, prioridad, cliente asociado y técnico asignado.
- Historial de tickets: seguimiento de cambios de estado, comentarios y fecha de modificación, vinculados al ticket y al usuario que realizó la acción.

La base de datos utilizada será MySQL, que permitirá manejar relaciones entre las tablas mediante claves primarias y foráneas. La capa de datos se encargará del acceso y persistencia de la información, asegurando integridad referencial y evitando que la lógica de negocio interactúe directamente con la base de datos.

4. TIPOS DE SERVICIOS

4.1 Servicio de utilidad

Se implementará un servicio de autenticación de usuarios, reutilizable en todo el sistema. Este servicio permitirá:

- Validar credenciales (correo y contraseña).
- Generar y administrar tokens de sesión para garantizar seguridad en las peticiones al backend.
- Controlar accesos según el rol del usuario (cliente, técnico, administrador).

Este servicio es transversal y puede ser utilizado por cualquier módulo de la aplicación, asegurando consistencia en la gestión de seguridad.

4.2 Servicio de entidad

Las entidades principales del sistema y sus atributos son:

a) Usuario

- id_usuario (INT)
- nombre (VARCHAR)
- correo (VARCHAR)
- contraseña (VARCHAR)
- rol (ENUM: administrador, técnico, cliente)
- fecha_creacion (DATETIME)
- id_creado_por (INT)
- Operaciones: crear usuario, actualizar datos, asignar rol, consultar información, eliminar usuario.

b) Cliente

- id_cliente (INT)
- telefono (VARCHAR)
- direccion (VARCHAR)
- empresa (VARCHAR)

- Operaciones: registrar cliente, actualizar datos, consultar información, eliminar cliente.

c) Ticket

- id_ticket (INT)
- titulo (VARCHAR)
- descripcion (TEXT)
- fecha_creacion (DATETIME)
- estado (ENUM: abierto, en proceso, cerrado)
- prioridad (ENUM: baja, media, alta)
- id_cliente (INT)
- id_tecnico (INT)
- id_creado_por (INT)
- Operaciones: crear ticket, modificar estado, asignar técnico, consultar ticket, cerrar ticket.

d) Historial de tickets

- id_historial (INT)
- id_ticket (INT)
- id_usuario (INT)
- fecha_cambio (DATETIME)
- estado_anterior (ENUM)
- estado_nuevo (ENUM)
- comentario (TEXT)
- **Operaciones:** registrar cambio de estado, consultar historial, añadir comentario.

4.3 Servicio de tarea

- Registro de ticket por el cliente: el cliente ingresa directamente su incidencia desde el frontend, evitando la intervención manual del técnico.
- Asignación automática de técnico: el sistema asigna un técnico según la categoría o prioridad del ticket.
- Cambio de estado del ticket: el técnico actualiza el estado (Abierto → En proceso → On Hold → Resuelto → Cerrado), generando automáticamente un registro en el historial.
- Notificación al usuario: se envía un correo o alerta al cliente cuando su ticket cambia de estado.
- Generación de reportes: el administrador puede obtener reportes de tickets por estado, prioridad o técnico asignado.

5. SEGURIDAD DEL SERVICIO WEB

El sistema implementa un modelo de seguridad basado en tres pilares fundamentales: autenticación, autorización y confidencialidad. Este enfoque garantiza la protección de la información y el acceso controlado a los recursos del sistema, alineándose con los principios de seguridad en arquitecturas orientadas a servicios (SOA).

5.1 Autenticación

La autenticación es el proceso mediante el cual el sistema valida la identidad del usuario. Para ello, se implementa un mecanismo basado en JSON Web Tokens (JWT) y cifrado de contraseñas con BCrypt.

Flujo de autenticación:

1. El usuario envía sus credenciales al endpoint: POST /api/v1/auth/login.
2. El sistema valida las credenciales comparando la contraseña cifrada con BCrypt.
3. Si las credenciales son correctas, se genera un token JWT que contiene el ID del usuario, rol y tiempo de expiración.
4. El cliente almacena el token.
5. En cada solicitud posterior, el cliente envía el token en el encabezado Authorization: Bearer <token>.

El token incluye un tiempo de expiración y está firmado digitalmente, lo que garantiza la seguridad y evita la manipulación de la información.

5.2 Autorización

Roles del sistema:

El sistema tendrá los siguientes roles de usuario:

Administrador (Jefe de Area):

Permisos: crear, leer, actualizar y eliminar cualquier recurso del sistema.

Acceso completo a la configuración, gestión de usuarios y generación de reportes.

Usuario estándar (Técnico):

Permisos: leer y actualizar únicamente sus propios datos.

Puede realizar operaciones básicas relacionadas con sus recursos asignados, pero no tiene acceso a la gestión de otros usuarios ni a la configuración global del sistema.

Usuario (Personal administrativo):

Permisos: acceso limitado a información pública o funcionalidad restringida.

No puede modificar datos ni acceder a información sensible.

Implementación de autorización:

Cada endpoint del servicio web verificará el rol del usuario incluido en el JWT antes de permitir el acceso a la operación solicitada.

Esto asegura que solo los usuarios con los permisos adecuados puedan realizar ciertas acciones, garantizando la integridad y seguridad del sistema.

5.3 Confidencialidad

La confidencialidad del servicio web se garantiza mediante el uso de protocolos de comunicación cifrados como HTTPS, lo que impide la interceptación de datos sensibles entre cliente y servidor. Las credenciales de acceso se almacenan de forma segura utilizando algoritmos de cifrado como bcrypt, evitando su exposición en caso de fuga de información. Además, los tokens JWT generados para la autenticación contienen únicamente datos mínimos (ID y rol), reduciendo el riesgo de filtración de información personal. Estas medidas aseguran que la información de los usuarios y de los tickets permanezca protegida frente a accesos no autorizados, fortaleciendo la confianza en el sistema y preservando la integridad de los datos.

5.4 Disponibilidad

La disponibilidad del sistema se asegura mediante un conjunto de prácticas orientadas a mantener la continuidad operativa. Se implementará un manejo controlado de errores, evitando caídas inesperadas y ofreciendo respuestas claras al usuario. Las peticiones inválidas o maliciosas serán filtradas para preservar la estabilidad del servicio. La infraestructura estará soportada en servidores confiables y escalables, como Tomcat junto con

MySQL lo que permitirá atender la demanda sin interrupciones. Por último, se contará con monitoreo constante mediante dashboards y alertas automáticas que faciliten la detección temprana de problemas y su resolución inmediata. Con estas acciones, se garantiza que el sistema esté disponible en todo momento, incluso bajo condiciones de alta carga o incidentes imprevistos.

6. Arquitectura de Software en Tiempo de Ejecución

La arquitectura de software en tiempo de ejecución permite describir cómo se comporta el sistema mientras está siendo utilizado por los usuarios. En el caso del sistema de gestión de tickets para SERPOST, el funcionamiento se basa en una arquitectura cliente-servidor, donde el frontend desarrollado en React se comunica constantemente con el backend en Spring Boot mediante servicios REST y respuestas en formato JSON.

Mientras el sistema está en ejecución, varios usuarios podrán acceder al mismo tiempo para registrar incidencias, consultar tickets, actualizar estados o visualizar reportes. Debido a ello, existen componentes que pueden modificarse dinámicamente sin afectar el funcionamiento del sistema y otros que requieren detener temporalmente el servicio para evitar errores o problemas de estabilidad.

Además, se consideran tiempos de respuesta aproximados durante la operación del sistema. Por ejemplo, el registro de un ticket debería realizarse en menos de 3 segundos y las consultas al dashboard tendrían un tiempo de respuesta estimado entre 2 y 5 segundos, dependiendo de la cantidad de usuarios conectados y del volumen de información almacenada.

6.1 Arquitectura Rígida

La arquitectura rígida del sistema corresponde a aquellos componentes críticos que no pueden modificarse sin detener temporalmente el servicio o reiniciar el servidor. Esto ocurre debido a que ciertos cambios afectan directamente la estructura principal del sistema, la seguridad o la estabilidad general de la aplicación.

En el sistema de tickets de SERPOST, los siguientes componentes forman parte de la arquitectura rígida:

Configuración del backend y servicios REST

Cualquier modificación en la lógica principal del backend desarrollado en Spring Boot requiere recompilar y reiniciar el servidor de aplicaciones. Por ejemplo:

- Cambios en los controladores REST.
- Modificación de endpoints.
- Alteración de reglas de negocio críticas.
- Cambios en la autenticación JWT.
- Actualización de dependencias del sistema.

Estas operaciones requieren detener temporalmente el servicio para evitar inconsistencias durante la ejecución.

Estructura de la base de datos

Los cambios estructurales en MySQL también representan una arquitectura rígida. Por ejemplo:

- Creación o eliminación de tablas.
- Modificación de claves primarias o foráneas.
- Cambios en relaciones entre entidades.
- Alteración de tipos de datos.

Estas modificaciones necesitan mantenimiento programado debido a que podrían afectar la integridad referencial y provocar fallos en consultas activas si se realizan mientras el sistema está en funcionamiento.

Configuración de seguridad

Los mecanismos de autenticación y autorización basados en JWT y BCrypt también forman parte de la arquitectura rígida. Cambios como:

- Modificación de políticas de seguridad.
- Cambio de algoritmos de cifrado.
- Configuración de tokens.
- Actualización de certificados HTTPS.

Requieren reinicio del servidor para garantizar que todas las sesiones y procesos utilicen la nueva configuración de forma segura.

Infraestructura del servidor

Las actualizaciones del servidor Tomcat, configuración de puertos, memoria asignada o despliegue de nuevas versiones del sistema también requieren detener temporalmente el servicio. Estas acciones normalmente se realizan durante ventanas de mantenimiento para minimizar el impacto sobre los usuarios.

En promedio, una actualización de este tipo podría requerir entre 5 y 15 minutos de indisponibilidad dependiendo del nivel de complejidad del cambio implementado.

6.2 Arquitectura Flexible

La arquitectura flexible corresponde a los componentes y funcionalidades que pueden modificarse dinámicamente mientras el sistema continúa operativo, sin necesidad de reiniciar el servidor ni interrumpir el servicio.

El sistema de gestión de tickets incorpora diferentes capacidades de flexibilidad que permiten una administración más eficiente y adaptable.

Gestión dinámica de tickets

Los usuarios y técnicos pueden interactuar con los tickets en tiempo real mientras el sistema permanece activo. Entre las operaciones flexibles se incluyen:

- Crear tickets.
- Actualizar estados.
- Asignar técnicos.
- Añadir comentarios.
- Consultar historial.
- Modificar prioridades.

Estas acciones se reflejan inmediatamente en la base de datos y son visibles para los usuarios sin afectar el funcionamiento general del sistema.

Administración de usuarios y roles

El administrador puede gestionar usuarios dinámicamente mediante el panel administrativo, permitiendo:

- Crear nuevas cuentas.
- Cambiar roles.
- Bloquear usuarios.
- Actualizar información de contacto.
- Restablecer contraseñas.

Todos estos cambios se aplican en tiempo real y no requieren reiniciar el backend.

Monitoreo y dashboards en tiempo real

El dashboard del sistema permite visualizar continuamente:

- Tickets abiertos.
- Tickets resueltos.
- Cumplimiento de SLA.
- Tiempo promedio de atención.
- Carga de trabajo de técnicos.

La información se actualiza automáticamente mediante consultas periódicas al backend, permitiendo supervisar el estado operativo del sistema en tiempo de ejecución.

Notificaciones automáticas

El sistema puede enviar notificaciones y alertas automáticamente sin interrumpir el servicio, por ejemplo:

- Avisos de cambio de estado.
- Alertas de vencimiento de SLA.
- Confirmación de creación de tickets.
- Notificaciones de cierre de incidencias.

Estas operaciones funcionan de manera asíncrona y permiten mantener la continuidad operativa.

Escalabilidad operativa

Gracias a la arquitectura REST y al desacoplamiento entre frontend y backend, el sistema permite agregar nuevos módulos o funcionalidades de manera progresiva, reduciendo el impacto sobre el servicio en producción.

Por ejemplo, podrían añadirse futuras funcionalidades como:

- Chat en tiempo real.
- Integración con correo corporativo.
- Reportes avanzados.
- Inteligencia artificial para clasificación automática de tickets.

En términos de tiempo de ejecución, las operaciones flexibles del sistema tienen tiempos de respuesta estimados entre 1 y 5 segundos, permitiendo una interacción continua y eficiente para los usuarios finales.

6.3 Arquitectura Adaptativa (Dinámica)

Indicación: Explica si el sistema tiene capacidad de "auto-gestión". Por ejemplo, si el servidor detecta que hay miles de personas conectadas y automáticamente activa más memoria o recursos para no caerse. Es la capacidad del software para ajustarse solo al entorno.

7. Modelos de Distribución y Licenciamiento

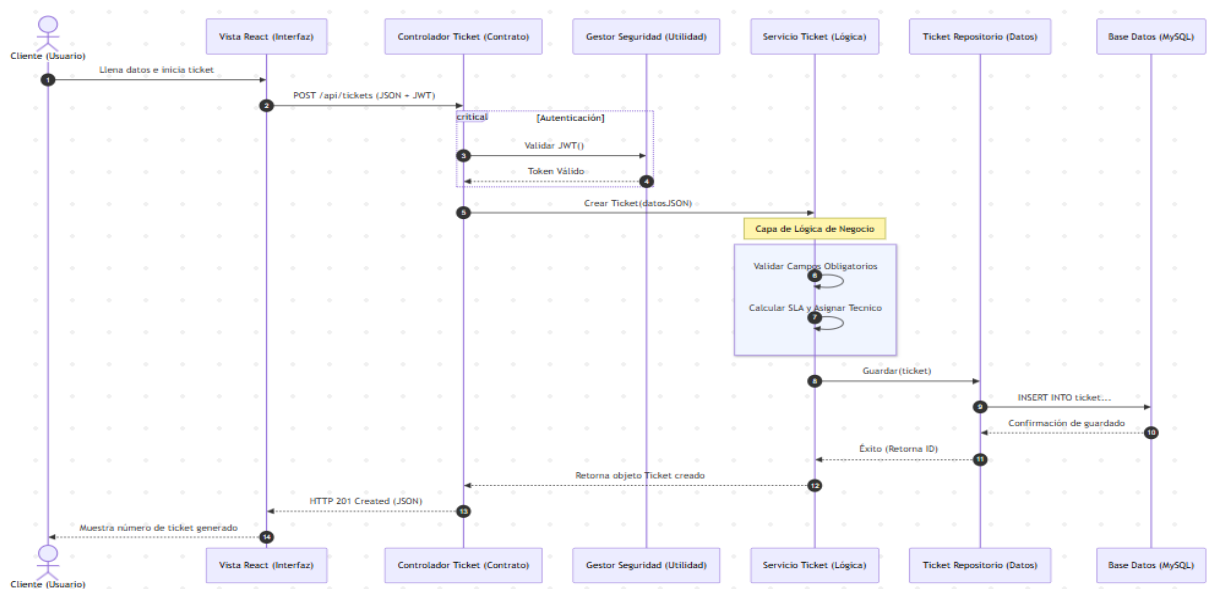
1. Restricciones por Rol: Usuario Gratuito vs. Usuario Premium.
 - Para incentivar la adquisición de licencias comerciales y segmentar las capacidades de la plataforma, se definen las siguientes restricciones según el nivel de suscripción:

1. Restricciones por Rol: Define qué hace un "Usuario Gratuito" vs un "Usuario Premium".
2. Usuario Gratuito (Mesa de Ayuda Básica): Tiene acceso limitado al sistema. Permite que los usuarios finales registren tickets de forma autónoma a través del Portal Usuario. Sin embargo, la asignación de técnicos debe realizarse de forma manual desde la bandeja de entrada, el historial de auditoría de los tickets está restringido a un periodo corto de tiempo y solo se permite registrar un número limitado de técnicos de soporte en el sistema.
3. Modelo Freemium: Explica que el sitio tiene funciones básicas abiertas (páginas de consulta), pero para usar las herramientas avanzadas (procesos de pago, reportes detallados), el usuario debe pagar una licencia de permiso.
4. Líneas de acceso: El sistema establece un flujo progresivo de acceso que regula cómo los usuarios interactúan con la plataforma, garantizando seguridad y control desde el primer contacto. En primer lugar, un invitado posee acceso limitado o nulo, restringido únicamente a páginas públicas de consulta. Esto asegura que no pueda manipular información sensible ni ingresar a recursos internos. Posteriormente, el usuario realiza el registro inicial, creando su entidad en la tabla usuarios. Tras la validación de identidad, el sistema activa la cuenta y asigna un rol específico. En este punto, el usuario pasa a ser registrado, con permisos para crear y gestionar tickets básicos, además de recibir notificaciones automáticas que informan sobre el estado de sus solicitudes. Finalmente, bajo el modelo freemium, el usuario registrado puede optar por convertirse en cliente con licencia premium, ampliando sus permisos mediante la adquisición de una suscripción comercial. Este acceso habilita funciones avanzadas como reportes detallados, procesos de pago y gestión extendida de recursos.

8. Diagramas Técnicos

8.1 Diagrama de Interacción (Secuencia):

El diagrama representa el ciclo de vida de una solicitud desde que el usuario interactúa con la pantalla hasta que los datos se consolidan de forma segura en el servidor.



Fase 1: Captura e Inicio de la Petición (Mensajes 1 y 2)

- 1. El Cliente inicia la acción: El usuario final (personal administrativo de SERPOST) interactúa con la interfaz gráfica completando el formulario con los detalles de su incidente (asunto, descripción, categoría) y hace clic en "ENVIAR TICKET".
- 2. Envío de datos (Petición HTTP): La Vista React empaqueta esta información en un formato ligero (JSON) y añade en las cabeceras el token de seguridad JWT que el usuario obtuvo al iniciar sesión. Esta petición viaja por la red mediante el método POST /api/tickets hacia el backend.

Fase 2: Filtro de Seguridad y Autenticación (Mensajes 3 y Envoltorio critical)

- 3. Validación del Token: El Controlador Ticket (el punto de entrada o endpoint en Spring Boot) recibe la solicitud. Antes de permitir que se ejecute cualquier lógica, delega la seguridad al Gestor Seguridad.
- Verificación: Este componente descifra el JWT para comprobar que no haya sido alterado, que esté vigente y que pertenezca a un usuario con rol autorizado (en este caso, un Cliente). Al confirmarse que es válido, el gestor le da luz verde al controlador para continuar.

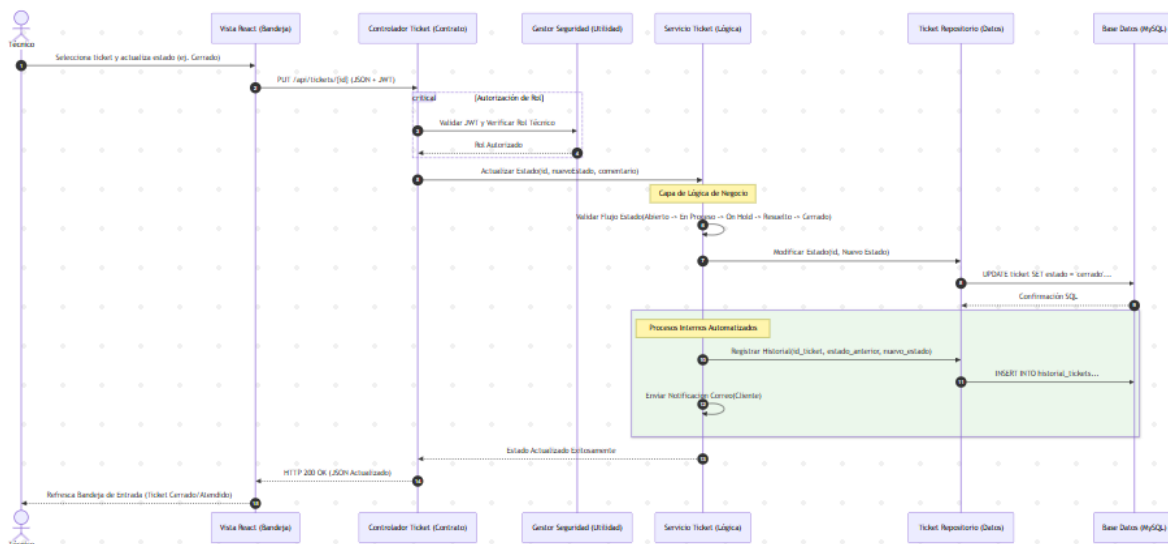
Fase 3: Procesamiento y Lógica de Negocio (Mensajes 4, 5, 6 y Envoltorio rect)

- 4. Delegación del procesamiento: Una vez autenticado, el controlador le transfiere el objeto JSON al Servicio Ticket, que es el verdadero "cerebro" de la aplicación donde residen las reglas operativas de la empresa.
- 5. Validar Campos Obligatorios (Auto-llamada): El servicio verifica internamente que los datos críticos no estén vacíos, por ejemplo, que la descripción y la categoría existan y sean coherentes.
- 6. Calcular SLA y Asignar Técnico (Auto-llamada): Aquí se ejecuta la automatización clave del proyecto. El sistema analiza la categoría seleccionada por el usuario (ej. Software, Hardware, Red) y, mediante un motor de reglas, determina la prioridad del ticket, calcula el SLA y asigna automáticamente un técnico disponible calificado para esa área.

Fase 4: Persistencia y Respuesta Final (Mensajes 7 al 10)

- 7 y 8. Almacenamiento en Base de Datos: El servicio envía la entidad de negocio completamente estructurada al Ticket Repositorio. Esta capa de persistencia traduce el objeto de Java en una sentencia SQL ejecutable (un INSERT INTO) y la envía a la base de datos MySQL.
- Confirmación: MySQL escribe el registro de manera permanente en el disco duro, genera un identificador único (ID numérico del ticket) y devuelve una señal de éxito.
- 9 y 10. Respuesta al usuario: El éxito del guardado viaja de regreso por las capas del backend hasta el controlador, el cual responde a la aplicación web con un estado HTTP 201 Created y el JSON del ticket finalizado. La VistaReact procesa esta respuesta y le muestra un mensaje interactivo en pantalla al Cliente con su número de ticket asignado como ejemplo: *"Ticket #105 registrado con éxito"*), cerrando el flujo de manera óptima.

Este diagrama de secuencia ilustra el flujo interactivo y automatizado cuando un técnico especialista de la Mesa de Ayuda de SERPOST atiende y actualiza el estado de una incidencia asignada. El proceso demuestra la interacción entre las capas de presentación, contrato, lógica de negocio y persistencia:



1. Petición y Control de Acceso (Pasos 1 al 3)

- Paso 1: El técnico interactúa con la interfaz gráfica en React (Vista React), selecciona un ticket de su bandeja y cambia su estado (por ejemplo, a *Cerrado*), adjuntando un comentario sobre la solución técnica aplicada.
- Paso 2: La interfaz web empaqueta estos datos en formato JSON y realiza una petición asíncrona hacia el backend utilizando el método PUT /api/tickets/{id}, adjuntando además el token JWT en las cabeceras.
- Paso 3 (Bloque critical): El Controlador Ticket intercepta la solicitud y acude al Gestor Seguridad. Aquí se ejecuta un filtro crítico de Autorización: el sistema valida la firma del JWT y verifica estrictamente que el usuario logueado posea el rol de Técnico o Administrador. Si un usuario con rol "Usuario" intentara alterar el estado, la petición sería rechazada inmediatamente.

2. Procesamiento de Lógica de Negocio (Paso 4 y Auto-llamada)

- Paso 4: Superado el filtro de seguridad, el controlador delega la ejecución al método Actualizar Estado dentro de la capa Servicio Ticket (Lógica de Negocio).
- Auto-llamada (Validar Flujo Estado): Antes de alterar la base de datos, el servicio ejecuta una regla de negocio esencial: verificar la transición del ciclo de vida del ticket. El sistema valida que el cambio respete la secuencia lógica e irreversible establecida: Abierto -> En Proceso -> On Hold -> Resuelto -> Cerrado. (Por ejemplo, impide que un ticket cerrado se vuelva a poner "en proceso" sin justificación).

3. Persistencia en la Base de Datos (Pasos 5 y 6)

- Pasos 5 y 6: Una vez aprobada la validación de estado, el servicio invoca al Ticket Repositorio (capa de datos). Este componente traduce la acción a una sentencia SQL estructurada y ejecuta un UPDATE ticket SET estado = 'cerrado'... directamente en la base de datos MySQL, devolviendo una confirmación de éxito tras impactar el registro.

4. Procesos Internos Automatizados (Bloque rect - Pasos 7 y 8)

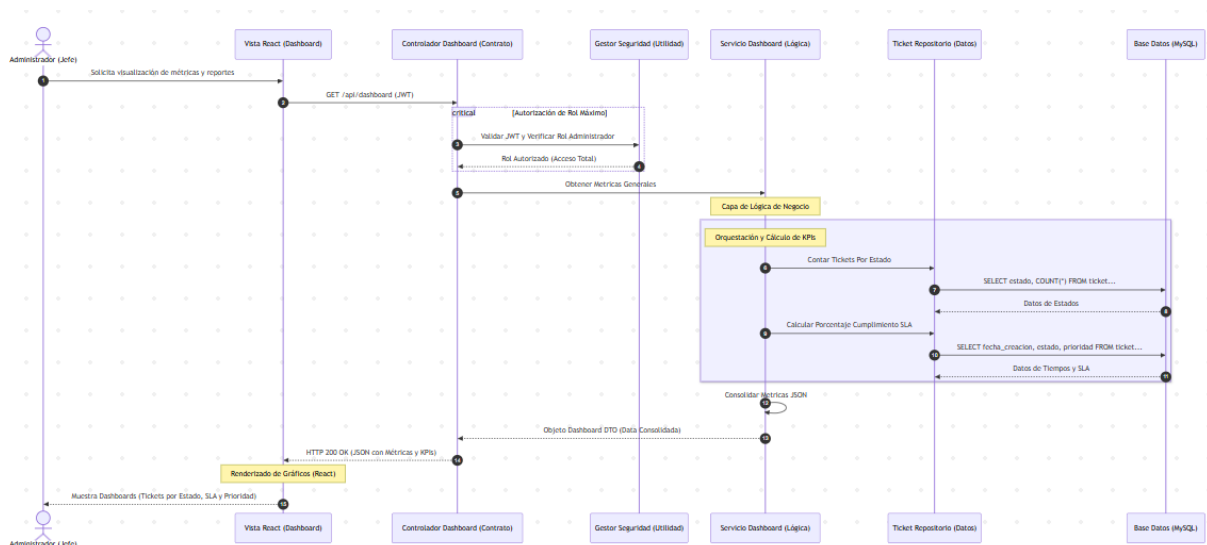
Una vez que el estado principal se actualiza, la lógica de negocio dispara en segundo plano dos automatizaciones cruciales para la trazabilidad y la transparencia del servicio:

- Paso 7 (Trazabilidad e Historial): El servicio ordena al repositorio registrar la acción en la tabla de auditoría. Se ejecuta un INSERT INTO historial_tickets en MySQL, almacenando de forma permanente el ID del ticket, el estado anterior, el nuevo estado, la fecha exacta y el usuario que realizó la modificación.
- Paso 8 (Comunicación): De forma paralela, el sistema invoca la función Enviar Notificación Correo (Usuario), despachando una alerta automatizada por correo electrónico al usuario final de SERPOST para informarle que su requerimiento ha sido solucionado.

5. Finalización del Flujo

- El Servicio Ticket confirma la operación exitosa al Controlador Ticket.
- El controlador responde a la aplicación web con un código de estado HTTP 200 OK y el JSON con los datos actualizados.
- La Vista React procesa la respuesta positiva y refresca la bandeja del técnico en tiempo real, moviendo o archivando el ticket atendido.

Este diagrama modela el flujo interactivo de extracción y procesamiento de datos analíticos cuando el Jefe de Área de TI ingresa al sistema para evaluar el rendimiento global de la mesa de ayuda de SERPOST.



Fase 1: Petición Analítica y Seguridad de Alto Nivel (Pasos 1 al 3)

- Paso 1: El Administrador ingresa al módulo de estadísticas en la interfaz web de React (Vista React (Dashboard)).
- Paso 2: La aplicación realiza una petición mediante el método GET /api/dashboard, enviando en la cabecera el token JWT de la sesión.
- Paso 3 (Bloque critical): El Controlador Dashboard intercepta la solicitud y acude al Gestor Seguridad. El componente valida que el token no haya expirado y comprueba estrictamente que el usuario posea el rol de Administrador (Jefe de Área). Al ser el rol con jerarquía máxima, se le otorga acceso total a las consultas agregadas del sistema.

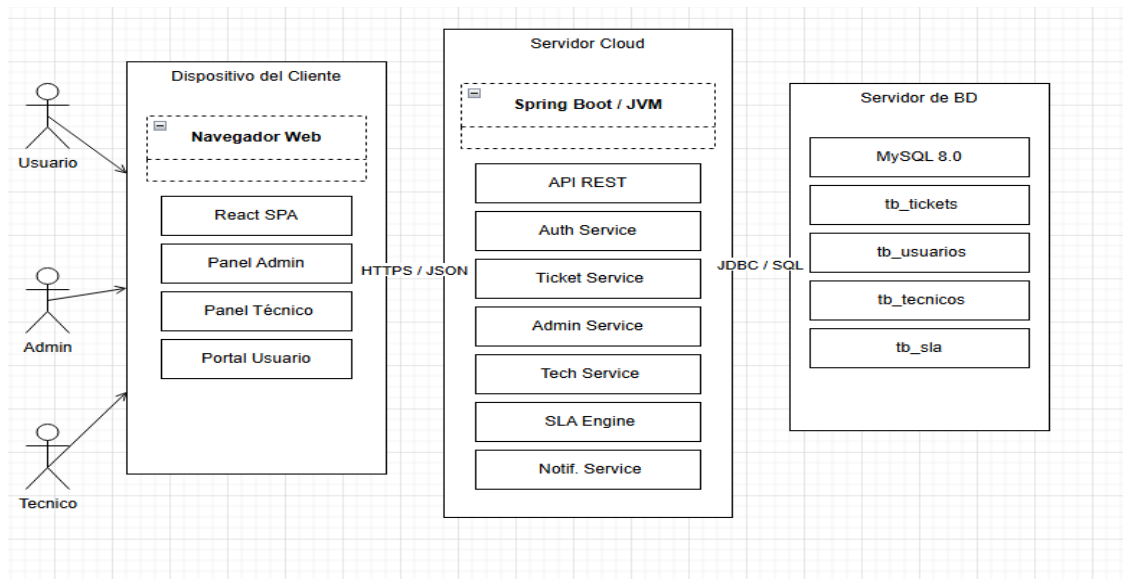
Fase 2: Orquestación de Consultas y Lógica Analítica (Paso 4 y bloque rect)

- Paso 4: Autorizado el acceso, el controlador invoca al componente Servicio Dashboard en la capa de Lógica de Negocio.
- Bloque de Orquestación de KPIs (rect): A diferencia de los flujos anteriores que solo afectaban a un ticket, el servicio del administrador debe coordinar múltiples consultas de agregación a la base de datos para construir los paneles informativos:
 - Paso 5 y 6: Llama al repositorio para realizar un conteo grupal de incidencias (SELECT estado, COUNT(*)... en MySQL), lo que alimentará el gráfico de Tickets por Estado de tu diseño de interfaz.
 - Paso 7 y 8: Solicita los datos de tiempos y criticidad para procesar matemáticamente el nivel de cumplimiento de los tiempos de atención establecidos (SLA).

Fase 3: Consolidación y Renderizado Dinámico (Pasos finales)

- Auto-llamada (Consolidar Métricas JSON): La lógica de negocio toma los resultados puros de las consultas, los procesa y los unifica en una estructura optimizada de transferencia de datos llamada DTO (Data Transfer Object) en formato JSON.
- El backend responde a la interfaz enviando un estado HTTP 200 OK cargado con toda la matriz de datos analíticos.
- Finalmente, los componentes avanzados de React procesan ese JSON de manera interactiva y transforman los números en los gráficos de barras y de pastel dinámicos que planificaron en sus maquetas (Balsamiq Wireframes), brindándole al administrador una visibilidad clara y en tiempo real para la toma de decisiones

8.2 Diagrama de Despliegue:



El diagrama representa tres nodos físicos interconectados y tres actores que interactúan con el sistema.

Actores

- **Usuario:** personal administrativo de SERPOST que registra y consulta tickets a través del Portal Usuario.
- **Admin:** administrador del sistema que gestiona configuraciones, usuarios y reportes desde el Panel Admin.
- **Técnico:** personal de soporte que atiende y resuelve los tickets asignados desde el Panel Técnico.

Los tres actores acceden al sistema a través del mismo nodo cliente.

Nodo 1 – Dispositivo del Cliente Representa el hardware del usuario final (PC o celular). Contiene un entorno de ejecución llamado Navegador Web, dentro del cual corre la aplicación React SPA. Adicionalmente aloja tres interfaces diferenciadas: Panel Admin, Panel Técnico y Portal Usuario, cada una adaptada al rol correspondiente.

Nodo 2 – Servidor Cloud Es el nodo central del sistema. Contiene el entorno de ejecución Spring Boot / JVM donde residen todos los servicios del backend: API REST (punto de entrada), Auth Service (autenticación JWT), Ticket Service (lógica principal), Admin Service (gestión administrativa), Tech Service (gestión de técnicos), SLA Engine (motor de reglas de niveles de servicio) y Notif. Service (envío de notificaciones y alertas).

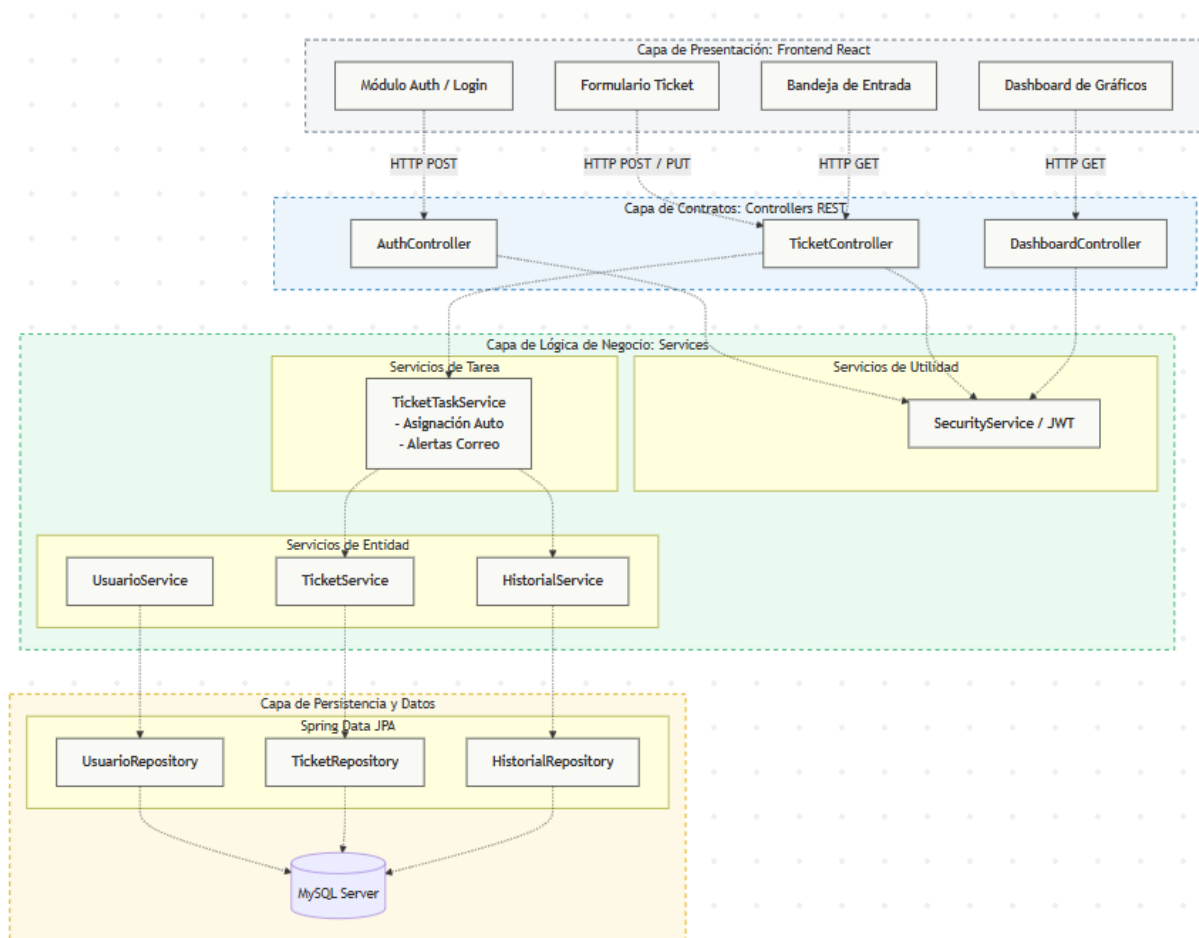
Nodo 3 – Servidor de BD Aloja la base de datos MySQL 8.0 con las tablas principales del sistema: tb_tickets, tb_usuarios, tb_tecnicos y tb_sla.

Comunicaciones

- Entre el Dispositivo del Cliente y el Servidor Cloud: protocolo HTTPS / JSON, garantizando cifrado y transporte ligero de datos.
- Entre el Servidor Cloud y el Servidor de BD: protocolo JDBC / SQL, para consultas y persistencia de datos.

8.3 Diagrama de Componentes:

El diagrama de componentes del Sistema de Tickets Automatizado para SERPOST adopta un patrón arquitectónico multicapa desacoplado. La estructura de los archivos y módulos de software se organiza de forma vertical y modular, asegurando el principio de alta cohesión y bajo acoplamiento. Las flechas discontinuas del diagrama (-->) representan relaciones de dependencia jerárquica, lo que significa que un componente de una capa superior requiere o consume los servicios de la capa inmediata inferior:



La organización se divide en cuatro capas principales:

1. Capa de Presentación (Frontend React)

Representa el lado del cliente (interfaz de usuario) y está fragmentada en módulos reutilizables especializados:

- **Módulo Auth / Login:** Bloque que gestiona el formulario de ingreso y encapsula las credenciales antes de ser enviadas.
- **Formulario Ticket:** Componente visual autónomo donde el usuario final registra las incidencias sin intermediarios.
- **Bandeja de Entrada:** Data Grid dinámico que lee el estado de las solicitudes y permite a los técnicos gestionar sus tareas.
- **Dashboard de Gráficos:** Componente que procesa la data analítica en tiempo real para el Administrador.

2. Capa de Contratos (Controllers REST - Spring Boot)

Es el punto de entrada al backend. Su única función es exponer los endpoints públicos definidos en el contrato del servicio y validar que la estructura de datos entrante en formato JSON sea correcta:

- **AuthController:** Gestiona la ruta `/api/login`.
- **TicketController:** Expone las rutas `/api/tickets` y `/api/tickets/{id}` para operaciones CRUD.
- **DashboardController:** Provee la ruta `/api/dashboard` para la extracción de métricas.

Dependencia Crítica: Todos los controladores dependen horizontalmente de `SecurityService`. Antes de procesar cualquier solicitud, interceptan la petición para validar la firma y expiración del token JWT.

3. Capa de Lógica de Negocio (Services)

Aquí reside el cerebro del sistema, donde se aplican las reglas operativas de SERPOST. La lógica se dividió en tres tipos de bloques autónomos:

- A. Servicios de Utilidad (`SecurityService`): Es un componente transversal e independiente. Se encarga de descifrar tokens JWT y ejecutar los algoritmos de hashing BCrypt para el control de accesos por roles.

- B. Servicios de Tarea (TicketTaskService): Actúa como un orquestador de flujos complejos. Cuando se ejecuta una tarea de negocio como cerrar un ticket, este componente coordina en simultáneo múltiples acciones: le dice al servicio de entidad que cambie el estado, le dice al servicio de historial que audite el cambio y gatilla el proceso de envío de alertas por correo electrónico.
- C. Servicios de Entidad (UsuarioService, TicketService, HistorialService): Son componentes atómicos y especializados. Cada uno contiene las reglas y validaciones estrictas y exclusivas de su propio modelo de datos (por ejemplo, validar que un ticket no salte de Abierto a Cerrado sin pasar por En Proceso).

4. Capa de Persistencia y Datos

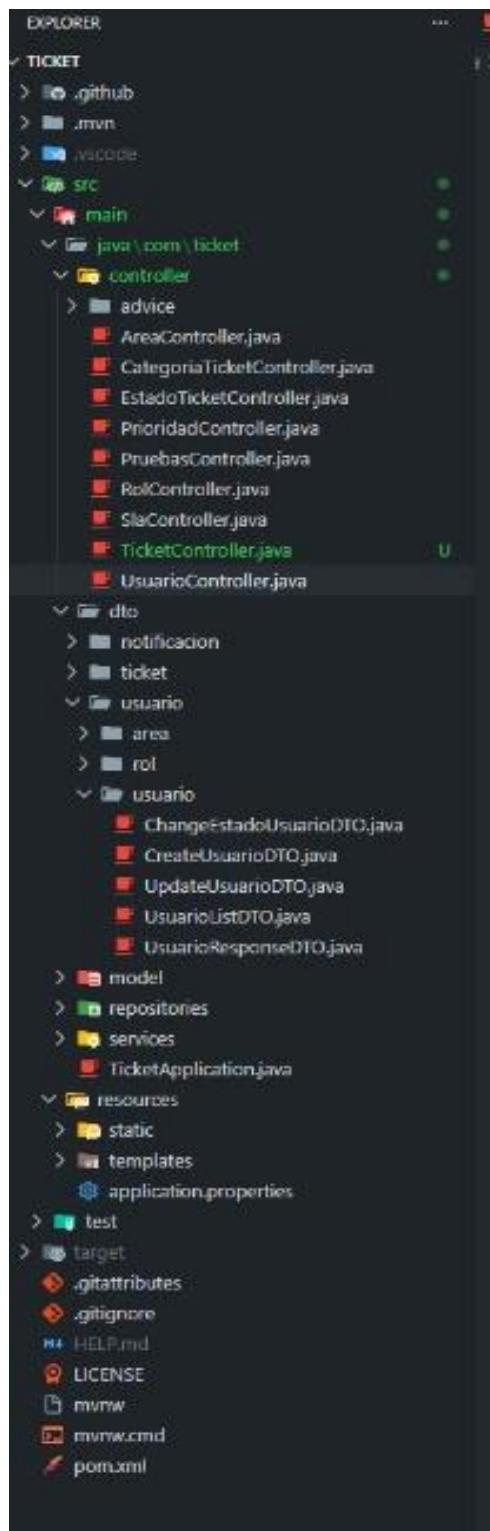
Es el nivel más bajo, encargado del acceso seguro a la información almacenada sin intervenir en las reglas de negocio:

- **Spring Data JPA (Repositories):** Interfaces espejo (UsuarioRepository, TicketRepository, HistorialRepository) que actúan como pasarelas de persistencia. Utilizan un mapeo objeto-relacional para traducir las entidades Java en sentencias SQL interactivas.
- **MySQL Server:** El motor relacional final donde residen físicamente las tablas y se garantiza la integridad referencial a través de claves primarias y foráneas.

9. Estado de Avance del Servicio Web (50%)

9.1. Evidencia de Código y Estructura:

- Muestra capturas de pantalla de la estructura de carpetas de tu proyecto (donde se vea la separación de la lógica, los modelos y los controladores).



- Captura de la definición del Contrato (archivo .yaml, .json o interfaz de código) donde se vean los métodos principales.


```

ticket
usuarioDTO.java UpdateUsuarioDB.java UsuarioController.java pom.xml X
> java > com > ticket > dto > usuario > usuario
package com.ticket.dto.usuario;

import jakarta.validation.constraints.NotNull;
import jakarta.validation.constraints.Size;

public class UpdateUsuarioDTO {

    private Long idUsuario;

    @NotNull(message = "El nombre es obligatorio")
    private String nombre;

    @NotNull(message = "El apellido es obligatorio")
    private String apellido;

    @NotNull(message = "El email es obligatorio")
    @Email(message = "Formato de email no válido")
    private String email;

    @NotNull(message = "El área es obligatoria")
    private Long idArea;

    @NotNull(message = "El rol es obligatorio")
    private Long idRol;

    @Override
    public String toString() {
        return "UpdateUsuarioDTO [idUsuario="
            + idUsuario + ", nombre="
            + nombre + ", apellido="
            + apellido + ", email="
            + email + ", idArea="
            + idArea + ", idRol="
            + idRol + "]";
    }

    public Long getIdUsuario() {
        return idUsuario;
    }

    public void setIdUsuario(Long idUsuario) {
        this.idUsuario = idUsuario;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getApellido() {
        return apellido;
    }

    public void setApellido(String apellido) {
        this.apellido = apellido;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public Long getIdArea() {
        return idArea;
    }

    public void setIdArea(Long idArea) {
        this.idArea = idArea;
    }

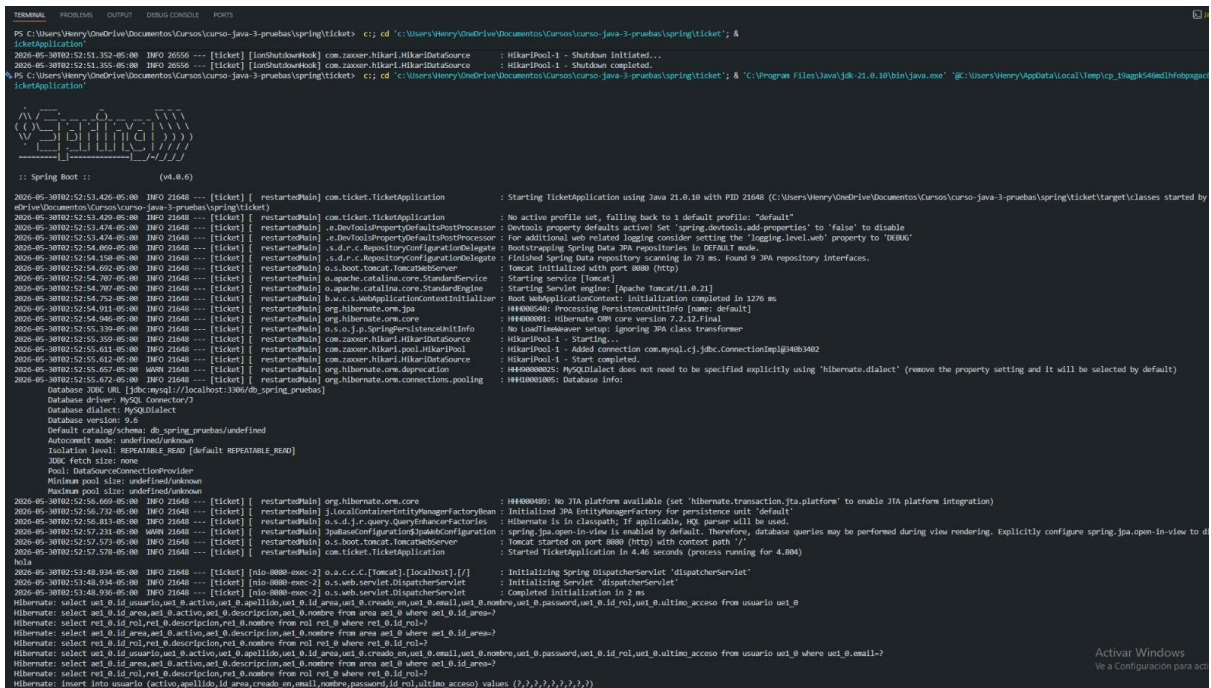
    public Long getIdRol() {
        return idRol;
    }

    public void setIdRol(Long idRol) {
        this.idRol = idRol;
    }
}
pom.xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/maven-v4_0_0/modelVersion">
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.0.5</version>
        <relativePath><!-- lookup parent from repository --></relativePath>
    </parent>
    <groupId>com.ticket</groupId>
    <artifactId>ticket</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>ticket</name>
    <description></description>
    <url></url>
    <licenses>
        <license></license>
    </licenses>
    <developers>
        <developer></developer>
    </developers>
    <scm>
        <connection></connection>
        <developerConnection></developerConnection>
        <tag></tag>
        <url></url>
    </scm>
    <properties>
        <java.version>21</java.version>
    </properties>
    <dependencies>Add Spring Boot Starters...
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
            <optional>true</optional>
        </dependency>
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
            <scope>test</scope>
        </dependency>
        <!-- Validaciones -->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-validation</artifactId>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
        </plugins>
    </build>
</project>
application.properties X
src > main > resources > application.properties
1 spring.application.name=ticket
2 spring.datasource.url=jdbc:mysql://localhost:3306/db_spring_pruebas
3 spring.datasource.username=root
4 spring.datasource.password=123
5 spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
6 spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
7 spring.jpa.show-sql=true
8
9 spring.jpa.hibernate.ddl-auto=update
10

```

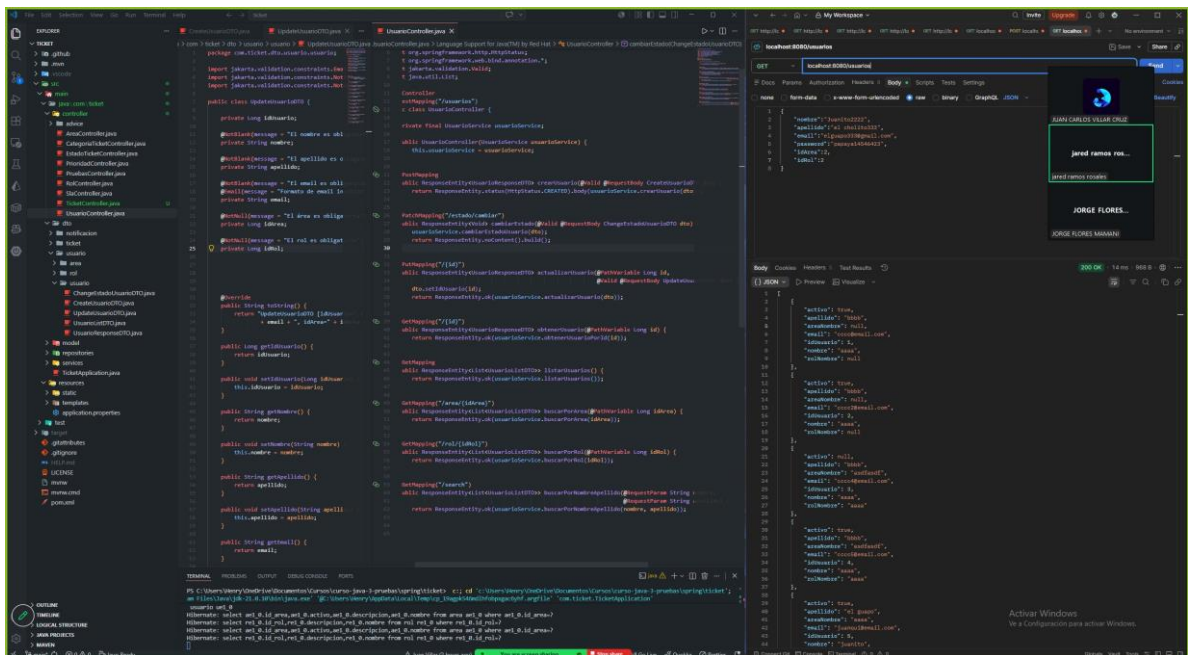
9.2. Pruebas de Funcionamiento (Capturas de Pantalla):

- Captura de Pantalla 1: El servicio web corriendo en el servidor local (localhost) o en la nube.



```
PS C:\Users\Henry\OneDrive\Documents\Cursos\curso-java-3-pruebas\spring\ticket> cd 'C:\Users\Henry\OneDrive\Documents\Cursos\curso-java-3-pruebas\spring\ticket'; &
ticketApplication
2020-05-30T02:52:51.332-05:00 INFO 20556 --- [ticket] [localhost] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown initiated...
2020-05-30T02:52:51.333-05:00 INFO 20556 --- [ticket] [localhost] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Shutdown completed.
PS C:\Users\Henry\OneDrive\Documents\Cursos\curso-java-3-pruebas\spring\ticket> cd 'C:\Users\Henry\OneDrive\Documents\Cursos\curso-java-3-pruebas\spring\ticket'; & 'C:\Program Files\Java\jdk-21.0.10\bin\java.exe' '%C:\Users\Henry\AppData\Local\Temp\cp_kag454ed\fbogag
ticketApplication'
:: Spring Boot ::
(v4.0.6)
2020-05-30T02:52:52.426-05:00 INFO 21648 --- [ticket] restartMain com.ticket.TicketApplication : Starting TicketApplication using Java 21.0.10 with PID 21648 (C:\Users\Henry\OneDrive\Documents\Cursos\curso-java-3-pruebas\spring\ticket\target\classes started by
eDrive\Documents\Cursos\curso-java-3-pruebas\spring\ticket)
2020-05-30T02:52:53.429-05:00 --- [ticket] restartMain com.ticket.TicketApplication : No active profile set, falling back to 1 default profile: 'default'
2020-05-30T02:52:53.434-05:00 --- [ticket] restartMain o.devtools.proxy.DefaultHostProcessor : Devtools property defaults active! Set 'spring.devtools.add-properties' to 'false' to disable
2020-05-30T02:52:53.434-05:00 INFO 21648 --- [ticket] restartMain o.devtools.proxy.DefaultHostProcessor : For additional web related logging consider setting the 'logging.level.web' property to 'DEBUG'
2020-05-30T02:52:54.009-05:00 INFO 21648 --- [ticket] restartMain o.s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2020-05-30T02:54:13.041-05:00 INFO 21648 --- [ticket] restartMain o.s.d.r.c.RepositoryConfigurationDelegate : Finished Spring data repository scanning in 73 ms. Found 9 JPA repository interfaces.
2020-05-30T02:52:54.692-05:00 INFO 21648 --- [ticket] restartMain o.s.boot.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2020-05-30T02:52:54.707-05:00 INFO 21648 --- [ticket] restartMain o.apache.catalina.core.StandardEngine : Starting service [Tomcat]
2020-05-30T02:52:54.707-05:00 INFO 21648 --- [ticket] restartMain o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/11.0.21]
2020-05-30T02:52:54.732-05:00 INFO 21648 --- [ticket] restartMain b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: Initialization completed in 1276 ms
2020-05-30T02:52:54.911-05:00 INFO 21648 --- [ticket] restartMain org.hibernate.orm.core : #####0000: Processing PersistenceUnitInfo [name: default]
2020-05-30T02:52:54.946-05:00 INFO 21648 --- [ticket] restartMain org.hibernate.orm.core : #####0000: Hibernate ORM core version 7.2.12.Final
2020-05-30T02:52:55.330-05:00 INFO 21648 --- [ticket] restartMain o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup; ignoring JPA class transformer
2020-05-30T02:52:55.350-05:00 INFO 21648 --- [ticket] restartMain com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2020-05-30T02:52:55.611-05:00 INFO 21648 --- [ticket] restartMain com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Added connection com.mysql.cj.jdbc.ConnectionImpl@40b3402
2020-05-30T02:52:55.613-05:00 INFO 21648 --- [ticket] restartMain com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
2020-05-30T02:52:55.621-05:00 INFO 21648 --- [ticket] restartMain org.hibernate.orm.connection : #####0000: HSQL dialect does not need to be specified explicitly using 'hibernate.dialect' (remove the property setting and it will be selected by default)
2020-05-30T02:52:55.672-05:00 INFO 21648 --- [ticket] restartMain org.hibernate.orm.connection : #####0000: Database info:
Database: XXXX url: jdbc:mysql://localhost:3306/db_spring_pruebas
Database driver: MySQL Connector/J
Database version: 9.0
Default catalog/schema: db_spring_pruebas/undefined
Autocommit mode: undefined/unknown
Isolation level: REPEATABLE_READ (default REPEATABLE_READ)
XXX fetch size: none
Pool: DataSourceConnectionProvider
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2020-05-30T02:52:56.609-05:00 INFO 21648 --- [ticket] restartMain org.hibernate.orm.core : #####0000: No JPA platform available (set 'hibernate.transaction.jpa.platform' to enable JPA platform integration)
2020-05-30T02:52:56.732-05:00 INFO 21648 --- [ticket] restartMain j.localContainerEntityManagerFactoryBean : Initializing JPA EntityManagerFactory for persistence unit 'default'
2020-05-30T02:52:56.813-05:00 INFO 21648 --- [ticket] restartMain o.s.d.r.c.query.QueryEnhancerFactories : Hibernate is in classpath; if applicable, HQL parser will be used.
2020-05-30T02:52:57.221-05:00 INFO 21648 --- [ticket] restartMain org.springframework.orm.jpa.persistence : Spring Jpa.open-in-view is enabled by default; therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to o
2020-05-30T02:52:57.579-05:00 INFO 21648 --- [ticket] restartMain com.ticket.TicketApplication : Tomcat started on port 8080 (http) with context path '/'
2020-05-30T02:52:57.579-05:00 INFO 21648 --- [ticket] restartMain com.ticket.TicketApplication : Started TicketApplication in 4.46 seconds (process running for 4.804)
2020-05-30T02:52:58.934-05:00 INFO 21648 --- [ticket] [info-0000-exec-2] o.a.c.c.c.[tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2020-05-30T02:53:48.934-05:00 INFO 21648 --- [ticket] [info-0000-exec-2] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2020-05-30T02:53:48.934-05:00 INFO 21648 --- [ticket] [info-0000-exec-2] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms
Hibernate: select uet.0.id,usuario,uet.0.activo,uet.0.apellido,uet.0.id_area,uet.0.creado,uet.0.email,uet.0.nombre,uet.0.password,uet.0.id rol,uet.0.ultimo_acceso from usuario uet 0
Hibernate: select uet.0.id,uet.0.rol,uet.0.descripcion,uet.0.nombre from area uet.0 where uet.0.id_area=?
Hibernate: select uet.0.id,uet.0.rol,uet.0.descripcion,uet.0.nombre from area uet.0 where uet.0.id_area=?
Hibernate: select uet.0.id,uet.0.rol,uet.0.descripcion,uet.0.nombre from area uet.0 where uet.0.id_area=?
Hibernate: select uet.0.id,uet.0.rol,uet.0.descripcion,uet.0.nombre from area uet.0 where uet.0.id_area=?
Hibernate: select uet.0.id,uet.0.rol,uet.0.descripcion,uet.0.nombre from area uet.0 where uet.0.id_area=?
Hibernate: select uet.0.id,uet.0.rol,uet.0.descripcion,uet.0.nombre from area uet.0 where uet.0.id_area=?
Hibernate: insert into usuario (activo,apellido,id_area,creado,et,0.email,nombre,password,id_rol,ultimo_acceso) values (?,?,?,?,,?,,?,,?)
Activar Windows
Ve a Configuración para activar Windows
```

- Captura de Pantalla 2: Una prueba de petición y respuesta. Si usas una herramienta como Postman o Insomnia, muestra cómo envías un dato y cómo el servicio responde correctamente (Código 200 OK).



```
POST http://localhost:8080/usuarios
{
  "nombre": "Jorge Flores",
  "apellido": "Flores",
  "email": "jorge.flores@ejemplo.com",
  "password": "123456",
  "rol": "ADMIN"
}

200 OK
{
  "mensaje": "Usuario creado exitosamente",
  "usuario": {
    "id": 1,
    "nombre": "Jorge Flores",
    "apellido": "Flores",
    "email": "jorge.flores@ejemplo.com",
    "password": "123456",
    "rol": "ADMIN",
    "activo": true,
    "ultimo_acceso": null
  }
}
```

- Captura de Pantalla 3: Conexión a la base de datos. Muestra una tabla con al menos un par de registros de prueba que el servicio esté utilizando.

9.3. Descripción del avance:

- Redacta un breve párrafo indicando qué funciones ya están terminadas y qué falta por implementar para llegar al 100%.

10. CONCLUSIONES

En este primer avance del proyecto, se ha logrado establecer con éxito la base estructural, técnica y estratégica para la modernización de la Mesa de Ayuda de la empresa SERPOST. En primer lugar, mediante un diagnóstico exhaustivo de la problemática operativa, se identificó que el modelo de registro manual por parte de los técnicos representaba una barrera crítica para la eficiencia institucional. Por consiguiente, la propuesta de implementar una arquitectura basada en servicios web (SOA) no surge solo como una mejora técnica, sino como una necesidad estratégica para garantizar la escalabilidad y la disponibilidad de los servicios de TI en una organización con alta demanda logística.

Asimismo, la importancia del diseño del servicio web radica en la creación de un contrato de interfaz claro y desacoplado. Gracias a esto, la interacción entre el *backend* desarrollado en Java (Spring Boot) y el *frontend* en React permite una separación de responsabilidades que facilita el mantenimiento preventivo y correctivo del software. La adopción del estilo arquitectónico REST y el formato JSON para el intercambio de datos asegura que el sistema sea ligero, rápido y compatible con futuros módulos móviles o integraciones con otros sistemas internos de SERPOST. Además, el diseño de la lógica de negocio garantiza que los niveles de servicio (SLA) se cumplan de forma automatizada, eliminando el error humano en la priorización de incidencias.

En el ámbito de la seguridad, la implementación proyectada de JSON Web Tokens (JWT) y el cifrado mediante BCrypt representa un salto cualitativo en la protección de los activos de información. De este modo, se garantiza que la trazabilidad de los tickets sea íntegra y que el acceso a la información sensible del cliente esté restringido según roles jerárquicos estrictos. La confidencialidad y disponibilidad, pilares de la norma ISO/IEC 27001, han sido integradas desde esta fase de diseño inicial, lo que reduce significativamente los riesgos de vulnerabilidades en etapas posteriores del ciclo de vida del desarrollo.

Finalmente, el desarrollo de este avance permite al equipo contar con una hoja de ruta técnica sólida, habiendo definido ya las entidades de datos, los procesos internos y los mecanismos de autorización. En consecuencia, el proyecto se encuentra plenamente preparado para las fases de construcción de la base de datos en MySQL y la codificación de los microservicios. Esta planificación rigurosa asegura que la solución final no solo resuelva la lentitud operativa actual, sino que también eleve la percepción de calidad del usuario final, transformando el soporte técnico en un motor de valor para la empresa.

11. BIBLIOGRAFÍA

1. Al-Hawari, F., & Barham, H. (2021). *A machine learning based help desk system for IT service management. Journal of King Saud University - Computer and Information Sciences*, 33(6), 702–718.
2. Zangari, A., Marcuzzo, M., Schiavinato, M., Gasparetto, A., & Albarelli, A. (2023). Ticket automation: An insight into current research with applications to multi-level classification scenarios. *Expert Systems with Applications*, 225, 119984.
3. Wibisono, Y., Wardhana, M. A., & Pamungkas, W. H. (2022). Design of Google Apps-based helpdesk ticketing system at university using extreme programming approach. *AIP Conference Proceedings*, 2658(1).